Codebase AI Engineer

AI-Powered Repository Intelligence & RAG-Based Code Understanding Engine

---

INTERVIEW PREPARATION GUIDE

A complete walkthrough of the project — what it does, how it works internally, the full tech stack, architecture, scaling roadmap, real-world use cases, and a recruiter-style Q&A bank.

Personal / Portfolio Project
Prepared August 2026

1 The 60-Second Pitch (memorize this)

Use this as your opening answer to "tell me about this project." Keep it to 60–90 seconds, then let the interviewer steer you deeper.

“Codebase AI Engineer is a tool that lets a developer point it at any codebase — a local folder or a public GitHub URL — and then ask natural-language questions about it and get semantic code search, instead of manually grepping through thousands of files. Under the hood it's a RAG pipeline: I scan the repository, parse Python files with the built-in AST module into functions, classes and methods, and fall back to a sliding-window line-based chunker for every other language. Each chunk gets a small context header — file path, language, imports, parent class, docstring — prepended before embedding, because that dramatically improves retrieval quality. I embed every chunk with a SentenceTransformers model and store the vectors in Pinecone, along with metadata like file path and line numbers. When you ask a question, I embed the query the same way, do a cosine-similarity search filtered to the active repo, and feed the top-k code chunks into an LLM — Gemini, Groq, or OpenAI, with automatic fallback between them — with a prompt that forces it to cite sources using [1], [2] style citation markers, which I then map back to exact file-and-line-number references in the UI. The whole ingestion pipeline streams live progress back to the browser over Server-Sent Events while it runs in a background thread, so you can watch it scan, parse, embed, and upsert in real time.”

Why this project is a strong interview story

- It's a real, working **RAG (Retrieval-Augmented Generation) system** — one of the highest-signal things you can show right now, since RAG is central to how LLMs are used in production.
- It covers **AST-based static analysis, vector databases, embeddings, multi-provider LLM integration, async/concurrent backend design, and a modern React frontend** — genuinely full-stack AI engineering.
- It has **real engineering trade-offs you can discuss intelligently** (single-process SSE, generic chunking for non-Python languages, index-wide clear) — self-awareness here is a strong signal.
- It's **immediately demoable** — you can point it at any public GitHub repo live in the interview.

2 What the Project Actually Does

Codebase AI Engineer turns any codebase — a local folder or a public GitHub repository — into a queryable knowledge base. It has two core capabilities, both built on top of one shared ingestion pipeline:

Capability	What it does
AI Code Assistant (RAG chat)	Ask a natural-language question ('How does authentication work here?') and get an answer grounded in actual code, with clickable citations to the exact file and line numbers.
Semantic Code Search	Search by meaning/intent ('where do we handle errors?') instead of exact keyword matching — finds relevant code even if your search words never literally appear in it.

There's also a **Repository Explorer** (file tree with live stats) and a dual interface: a full web dashboard (React) and a CLI (`ingest` / `search` / `ask` / `server` commands) for people who prefer the terminal.

3 Why This Is Better Than the Obvious Alternatives

Alternative	Why it falls short
Ctrl+F / grep / GitHub's search bar	Purely literal keyword matching — misses code that does the right thing but doesn't use your exact search words. Semantic search finds by *meaning*, not just text.

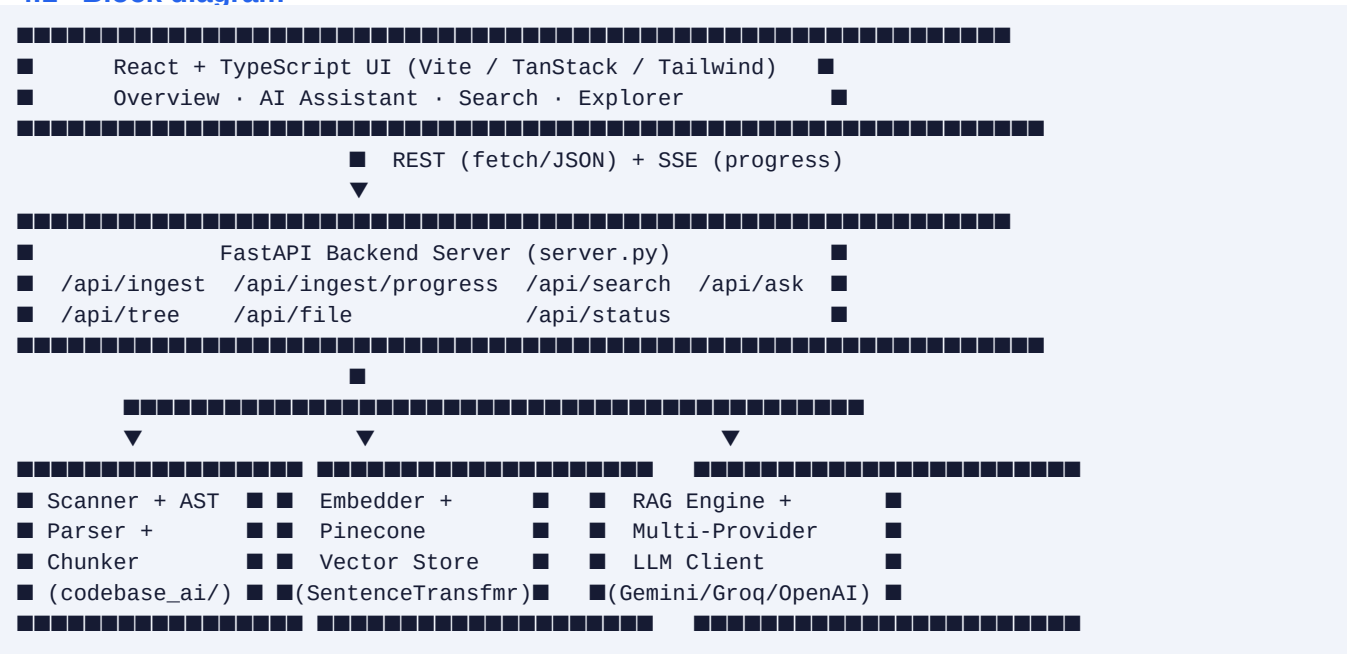
Alternative	Why it falls short
Pasting the whole repo into an LLM's chat window	Most repos don't fit in a context window at all, it's slow and expensive to resend every time, and it gives zero exact line-number citations — you get vague, potentially hallucinated answers you can't verify.
A generic AI coding assistant with no repo indexing	Has no persistent, searchable memory of your specific codebase — you have to keep re-explaining context every session, and it can't search across the whole repo at once.
Manually reading through unfamiliar code	Slow, and doesn't scale to legacy codebases with tens of thousands of files — this is exactly the 'onboarding a new engineer' pain point the project targets.

The core idea that makes this work well: retrieval-augmented generation grounds every LLM answer in *actual retrieved code*, with citations you can click and verify — instead of the LLM guessing from what it was trained on (which may not even include your private/internal code at all).

4 High-Level Architecture

The system has two halves: a **React frontend** (dashboard) talking over REST/SSE to a **FastAPI backend**, which orchestrates an ingestion pipeline and a query/RAG pipeline sharing the same vector store.

4.1 Block diagram



4.2 The two pipelines that matter most

Pipeline	Trigger	What happens
Ingestion pipeline	POST /api/ingest (repo path or GitHub URL)	Scan → Parse (AST) → Chunk + contextualize → Embed → Upsert to Pinecone, streaming progress the whole way
Query pipeline	POST /api/search or /api/ask	Embed the query → Vector similarity search in Pinecone (filtered to the active repo) → (for /ask) feed top-k chunks into an LLM with a citation-forcing prompt

5 What's Happening Internally, Step by Step

5.1 Ingestion — from a repo URL to searchable vectors

1. **Resolve the repo.** If it's a GitHub URL, RepoScanner tries three strategies in order: (1) the GitHub API tree endpoint (fetch the file tree, download only files with known source-code extensions individually), (2) downloading the repo as a zip archive and extracting it, (3) falling back to a shallow `git clone --depth 1`. If it's a local path, it's used directly. This tiered fallback means it works even in restricted network environments where one method might be blocked.
2. **Scan the directory.** Walks every folder, skipping ignored directories (.git, node_modules, .venv, __pycache__, etc.), ignored files, and ignored binary/media extensions. It also respects the repo's own .gitignore patterns. Every valid file becomes a CodeFile with its relative path, detected language, and raw text content.
3. **Parse into structural CodeUnits.** Python files go through PythonParser, which uses Python's built-in `ast` module to walk the syntax tree and emit one CodeUnit per class, function, and method — capturing exact

start/end line numbers, the parent class (for methods), the file's imports, and any docstring. Every other language falls back to GenericParser, a sliding-window line-based chunker (40 lines per chunk, 10-line overlap) since there's no AST parser wired up for those languages yet. Very large or binary-like files (over 500KB, or blacklisted extensions like .pkl/.onnx) get reduced to a tiny 'file_summary' unit — just the first 10 lines — so they don't waste embedding compute or blow up storage.

4 **Chunk + contextualize.** CodeChunker takes every CodeUnit and builds a small header before the actual code — file path, language, unit type, name, line range, parent class, imports, first line of the docstring — and prepends it to the code text before embedding. This is a deliberate RAG design choice: embedding the code *with context* makes retrieval far more accurate than embedding raw code alone, because the model can match on file/module intent, not just token overlap. Each chunk also gets a deterministic ID (an MD5 hash of file path + unit name + line range), so re-ingesting the same repo produces stable, idempotent IDs.

5 **Embed in batches.** CodeEmbedder wraps a SentenceTransformers model (all-MiniLM-L6-v2 by default, 384-dimensional vectors) and encodes the contextualized chunk text in batches of 64, emitting progress events after every batch so the frontend progress bar moves smoothly instead of jumping at the end.

6 **Upsert to Pinecone.** Every chunk's vector, ID, and metadata (file path, line numbers, unit name/type, imports, a repo_name tag, and the code text itself — truncated to 8,000 characters to respect Pinecone's per-vector metadata size limit) is upserted in batches of 100 into a Pinecone serverless index (cosine similarity metric).

7 **Stream progress the whole time.** Every step above calls an `emit()` callback with a step name, a human message, and a 0–100 percentage. The backend broadcasts each event to every connected browser over Server-Sent Events, so the UI's progress dialog updates live — scanning → parsing → embedding → upserting → done.

5.2 Answering a question — the RAG flow

1 The user's natural-language question is embedded using the exact same SentenceTransformers model used during ingestion — this consistency is essential, since the query vector and the code vectors must live in the same embedding space to be comparable.

2 PineconeStore.query() runs a cosine-similarity nearest-neighbour search, filtered by a metadata equality filter on `repo_name`, so a question always searches only the currently active repository, even if the shared index contains vectors from other repos indexed earlier.

3 The top-k matching chunks (each with file path, line range, unit name/type, and the actual code text) are formatted into numbered 'CITATION [n]' blocks and assembled into a single context string.

4 RAGEngine builds a system prompt instructing the model to behave as an expert software engineer, answer using the provided context, and cite sources using [n] bracket notation only when actually relevant — then sends it to LLMClient.

5 LLMClient tries providers in priority order — Gemini first, then Groq, then OpenAI — using whichever API key is present in the environment, and gracefully moving to the next provider if a call fails. If no provider is configured or all fail, a local heuristic fallback formats the top retrieved chunk directly as a readable answer, so the tool never just crashes with no answer.

6 The LLM's raw answer text is scanned with a regex for [n] citation markers; only the citations actually referenced in the answer are returned to the frontend — this keeps the citation list precise instead of dumping every retrieved chunk regardless of whether the model actually used it.

6 Tech Stack — What, and Why Each Piece Was Chosen

6.1 Backend

Technology	Role / why it was used
Python 3.10+	Best ecosystem for AST parsing, embeddings, and ML/AI tooling — every core library used (sentence-transformers, ast, pinecone SDK) is Python-native.
FastAPI + Uvicorn	The REST API layer. Chosen for native async support (needed for the SSE progress stream), automatic request validation via Pydantic models, and low boilerplate.
Python's built-in ast module	Powers the Python-specific parser — walks the real abstract syntax tree to find exact class/function/method boundaries, far more reliable than regex or naive line-splitting.
sentence-transformers	Generates dense vector embeddings (all-MiniLM-L6-v2, 384 dimensions) for both code chunks and user queries — a small, fast, locally-run model, so embedding doesn't depend on an external paid API.
Pinecone (serverless)	The vector database — stores every code chunk's embedding plus metadata, and serves fast cosine-similarity nearest-neighbour search at query time.
Multi-provider LLM client (Gemini / Groq / OpenAI)	Generates the final natural-language answer from retrieved context. Supporting three providers with automatic fallback means the tool keeps working even if one provider's free tier is exhausted or an API call fails.
Server-Sent Events (SSE) via asyncio.Queue	Streams live ingestion progress to the browser. An asyncio.Queue per connected client, fed by a background thread via loop.call_soon_threadsafe, lets a synchronous ingestion thread push updates into the async event loop safely.
threading (background ingest worker)	Ingestion is CPU/IO-heavy (scanning, parsing, embedding) and would block FastAPI's async event loop if run inline — so it runs on a background daemon thread, freeing the event loop to keep serving other requests.
Pydantic	Request/response schema validation for every API endpoint (IngestRequest, SearchRequest, AskRequest) — rejects malformed requests automatically before they reach business logic.
GitPython / raw urllib (repo fetching)	Used across the three-tier GitHub fetch strategy — GitHub API tree endpoint, zip archive download, and shallow git clone fallback.
Typer + Rich (CLI)	Powers the command-line interface (ingest/search/ask/server commands) with formatted tables, syntax-highlighted code panels, and progress panels — a genuinely usable terminal UX, not just a bare script.

6.2 Frontend

Technology	Role / why it was used
React 18 + TypeScript	Component-based UI with type safety across the whole frontend — catches integration bugs (e.g. mismatched API response shapes) at compile time.
Vite	Fast dev server and build tool — near-instant hot reload during development.
TanStack Router + TanStack Query	File-based routing between pages (Overview/Assistant/Search) and declarative data-fetching/caching for API calls, so components don't hand-roll loading/error/cache state.
Tailwind CSS + Radix UI primitives (shadcn/ui style)	Utility-first styling plus accessible, unstyled component primitives (dialogs, dropdowns, tooltips) — a fast way to build a polished dark dashboard without a heavy design-system dependency.

Technology	Role / why it was used
react-markdown	Renders the LLM's markdown-formatted answers (including citation links and code blocks) properly instead of showing raw markdown text.
recharts	Powers any chart/stat visualizations on the Overview dashboard (e.g. indexing stats).

Interview-ready one-liner on the stack choice: “I picked FastAPI specifically because ingestion needed to run in the background while the API kept responding, which meant I needed both a background thread for the heavy synchronous work and native async support for streaming live progress back over SSE — and I kept the frontend and backend fully decoupled behind a REST/SSE boundary so either side could be replaced independently.”

7 How the Backend Actually Runs

A single FastAPI process, started via `python server.py` (Uvicorn on 0.0.0.0:8000), serves everything — REST endpoints, the SSE progress stream, and file/tree reads — there's no separate microservice for any of it.

Concurrency model

- The main **asyncio event loop** handles all normal HTTP requests (search, ask, tree, file) — these are fast, request/response operations.
- When an ingest is triggered, a **background daemon thread** (`threading.Thread`) runs the actual scanning/parsing/embedding/upserting work — this is synchronous, potentially slow work that would otherwise block the event loop and freeze every other request.
- A module-level `threading.Event` (`_ingest_running`) acts as a simple lock — only one ingest can run at a time process-wide; a second request while one is running gets an HTTP 409.
- Progress events cross from the background thread into the async world via `loop.call_soon_threadsafe(queue.put_nowait, event)` — this is the exact mechanism that lets a plain synchronous thread safely push into an asyncio.Queue that the SSE endpoint is awaiting on.
- Each connected browser gets its **own asyncio.Queue**, registered on connect and cleaned up on disconnect, so multiple people can watch the same ingest's progress simultaneously, and late-joining clients immediately receive the last known progress event instead of seeing a blank screen.

Persistence (intentionally lightweight)

Indexing state (the latest stats, which repo is active) is persisted to a flat `index_state.json` file on disk, loaded on server startup — enough to survive a server restart in a single-instance, single-developer setup, but not a real database (a fair limitation to name proactively — see §11).

Multi-repo isolation, today

All repos share **one Pinecone index**; isolation between repositories is done purely by tagging every vector with a `repo_name` metadata field and filtering every query by it. This works, but it's a soft boundary, not a hard one — worth knowing as a discussion point on multi-tenancy.

8 Advantages

- **Grounded, verifiable answers** — every LLM response is backed by retrieved code with exact file/line citations, not free-floating text the user has to trust blindly.
- **Works on codebases larger than any LLM's context window** — because it retrieves only the relevant chunks instead of stuffing the whole repo into a prompt.
- **AST-aware chunking for Python** — chunks align with actual functions/classes/methods, not arbitrary line cuts, so retrieved context is always a coherent, complete unit.
- **Context-enriched embeddings** — the file/type/imports/docstring header prepended before embedding measurably improves retrieval relevance over embedding bare code.
- **Resilient LLM layer** — three-provider fallback (Gemini → Groq → OpenAI) plus a local heuristic fallback means the tool degrades gracefully instead of hard-failing when a provider is down or unconfigured.
- **Works on any public GitHub repo or local folder** — the tiered fetch strategy (API → zip → git clone) makes ingestion robust across different network/permission environments.
- **Real-time visibility into a slow operation** — SSE-based live progress turns an opaque, multi-minute indexing job into something the user can actually watch and trust.
- **Dual interface** — the same core pipeline is usable from a full web dashboard or a scriptable CLI.

9 Real-World Applications

Use case	How this project applies
New-engineer onboarding	Let a new hire ask natural-language questions about an unfamiliar internal codebase instead of tracking down the one person who remembers how a module works.
Legacy code modernization	Quickly locate every place a deprecated API/pattern is used across a large legacy codebase before a migration, using semantic search instead of brittle text search.
Code review assistance	Reviewers can ask 'what does this function actually do end-to-end' and get a grounded explanation with the exact call chain cited.
Technical due diligence (M&A;)	Quickly build an understanding of an acquisition target's codebase quality, architecture, and dependency structure without weeks of manual review.
Internal documentation generation	Use the RAG Q&A; as a base to auto-generate or keep 'living' architecture docs grounded in the real, current code rather than stale hand-written docs.
Security / compliance auditing	Semantically search for patterns like 'where do we validate user input' or 'where are secrets/credentials read' across an entire codebase.
Open-source learning	Point it at any public GitHub repo to explore and understand a library's internals interactively before contributing.
Internal developer chatbot / Slack integration	The REST API is a natural backend for a Slack/Teams bot that answers engineering questions grounded in the company's actual repos.

10 Limitations (know these — a recruiter will ask, and naming them yourself is a strength)

Limitation	Why it matters
Only Python gets real AST parsing	Every other language (JS, TS, Java, Go, etc.) falls back to a naive line-based sliding-window chunker — chunk boundaries can split a function in half, hurting retrieval quality for non-Python code.
clear_index() wipes the entire shared Pinecone index	Since multiple repos share one index, 're-index with clear' for one repo currently deletes every other indexed repo's vectors too — a real bug worth fixing before any multi-repo/multi-user use.
Single-process, in-memory SSE client list	Connected-client tracking (_client_queues) lives in one Python process's memory — this can't be horizontally scaled behind a load balancer without a shared pub/sub layer.
One global ingest lock	Only one repository can be indexed at a time, process-wide — no concurrent multi-user ingestion.
Flat-file state persistence (index_state.json)	Fine for a single local developer instance; not safe under concurrent writes or multiple server instances.
No authentication on API endpoints	Anything that can reach the FastAPI server can trigger ingestion or query — fine for a local tool, not production-ready as-is.
Full re-embedding on every ingest	There's no incremental/diff-based re-indexing — even a one-line change means re-scanning, re-parsing, and re-embedding the whole repository.
Chunk text truncated at 8,000 characters	Very large functions/classes get truncated in the stored metadata (a Pinecone per-vector metadata size constraint), which can lose context for huge code units.

How to frame these in an interview: pick one or two and say what you'd do about them — e.g. “clear_index() deleting the whole shared index instead of just the active repo's vectors is a real bug I'd fix by using repo-scoped delete-by-metadata-filter, or Pinecone namespaces per repo, instead of a global delete-all.”

11 Improvements & How to Scale This to Industry Level

11.1 Fixes and improvements to the current design

Area	Improvement
Multi-language AST parsing	Replace the generic line-based fallback with tree-sitter, which has real grammars for JS/TS/Java/Go/Rust/C++/etc. — giving every language the same function/class-aware chunking Python already gets.
Per-repo index isolation	Use a dedicated Pinecone namespace (or a metadata-scoped delete) per repository, so clearing one repo's index can never touch another's vectors.
Incremental re-indexing	Diff against the last indexed git commit (via git diff) and only re-parse/re-embed changed files, instead of a full rescan every time — massively cuts cost and time on large, frequently-updated repos.
Hybrid search (vector + keyword)	Combine the current dense vector search with a classic keyword index (BM25/Elasticsearch) — pure semantic search can miss queries about exact identifiers, error messages, or config keys.
Reranking step	Add a cross-encoder reranker on the top-k vector search results before they go to the LLM — meaningfully improves precision over raw embedding similarity alone.
Streaming LLM responses	Stream the LLM's answer token-by-token to the frontend instead of waiting for the full response — much better perceived latency for longer answers.

Area	Improvement
Conversation memory	Persist chat history per session so follow-up questions ('what about the async version of that?') retain context instead of each question being independent.
Authentication & per-repo access control	Add user accounts and repo-level permissions before this could be used on private/internal codebases by more than one trusted person.
Horizontally scalable SSE / real-time layer	Move connected-client tracking out of in-process memory into a shared pub/sub layer (Redis Pub/Sub) so progress streaming works correctly behind multiple backend instances/load balancers.
Dedicated task queue for ingestion	Replace the single background thread + global lock with a proper task queue (Celery/RQ/Arq) so multiple repos can be ingested concurrently, with retries and observability.

11.2 Scaling to an industry-grade product

1. **Multi-tenant SaaS model:** move from 'one shared index, filtered by repo_name' to proper tenant isolation — a Pinecone namespace (or a fully separate index for large customers) per organization, with auth-scoped API keys and per-tenant rate limits.
2. **Event-driven, always-fresh indexing:** instead of manual 'click to re-index,' register a GitHub webhook so every push automatically triggers an incremental re-index of just the changed files.
3. **Decouple ingestion from the API server:** move ingestion into its own worker pool/service consuming from a queue (Kafka/SQS/Redis Streams), so the API server stays lightweight and ingestion scales independently based on queue depth.
4. **Cache hot queries:** common questions asked repeatedly against a repo (especially right after a big team starts using it) can be cached, cutting both LLM cost and latency.
5. **Swap to a code-specialized embedding model** as usage grows — general-purpose sentence embeddings (like all-MiniLM) work well as a starting point, but a model trained specifically on code (e.g. a code-embedding model) typically retrieves more precisely at scale.
6. **Observability:** track ingestion latency per repo size, retrieval hit-rate, LLM cost per query, and citation-click-through (a good proxy for 'was this answer actually useful') — the kind of metrics a real product team would want before scaling spend.
7. **Cost control on LLM calls:** route cheap/simple queries to a smaller/faster model and only escalate to a larger model for complex multi-hop questions — a common industry cost-optimization pattern for RAG systems at scale.

12 Interview Question Bank (Recruiter-Style, With Answers)

12.1 Project Overview / Behavioral

Q: Tell me about this project in your own words.

A: Use the 60-second pitch in §1, then pause and let the interviewer steer with a follow-up.

Q: What was the hardest part of building this?

A: Getting the concurrency model right — ingestion is slow, synchronous work (scanning/parsing/embedding), but FastAPI's request handling is async, and I needed live progress streamed to the browser the whole time. I ended up running ingestion on a background thread and bridging progress events into the asyncio event loop with loop.call_soon_threadsafe so a synchronous thread could safely push into an async SSE queue without blocking anything.

Q: What would you do differently if you started over?

A: I'd build the multi-language chunking with tree-sitter from day one instead of a line-based fallback for non-Python languages — right now Python gets meaningfully better retrieval quality than every other language because only it gets real AST-aware chunking.

Q: How is this different from just using GitHub Copilot Chat or ChatGPT with code pasted in?

A: Those tools don't have a persistent, searchable index of your whole repository — you're limited by the context window and have to keep re-supplying code manually. This project builds a vector index once, so any later question can retrieve exactly the relevant few chunks out of potentially thousands of files, with citations pointing back to exact file and line numbers, which those chat tools generally don't give you.

12.2 RAG / Retrieval / Embeddings

Q: Walk me through what happens end-to-end when a user asks a question.

A: The query text is embedded with the same SentenceTransformers model used at ingestion time. That query vector is compared against every stored chunk vector in Pinecone using cosine similarity, filtered to the active repo. The top-k matches — each with file path, line numbers, and code text — are formatted into numbered citation blocks and passed to the LLM in a prompt that instructs it to answer using only that context and cite sources with [n] markers. The response is scanned for which citation numbers were actually used, and only those are shown to the user.

Q: Why prepend a context header (file path, imports, docstring) before embedding, instead of embedding raw code?

A: Because embedding models pick up on more than just token overlap — giving them the file path, the unit type, imports, and docstring alongside the code gives the embedding richer signal about what the code is actually for, which measurably improves how well a natural-language question matches the right chunk. It's a well-known RAG best practice: enrich what you embed, don't embed raw content in isolation.

Q: Why use AST parsing instead of just splitting files into fixed-size chunks everywhere?

A: Fixed-size chunking can cut a function or class in half, which both breaks the LLM's ability to understand it and hurts embedding quality, since the chunk no longer represents one coherent unit of meaning. AST parsing for Python guarantees every chunk is a complete, semantically meaningful unit — a whole function, method, or class — which is why Python chunks retrieve noticeably better than the generic line-based chunks used for other languages today.

Q: Why SentenceTransformers/all-MiniLM instead of a bigger embedding model or an OpenAI embedding API?

A: It runs locally, it's fast, it's free (no per-call API cost or external dependency for something that runs on every single indexed chunk), and at 384 dimensions it's small enough to keep the Pinecone index compact. For a project at this scale it hits a good accuracy-versus-cost-and-latency balance; a production system indexing millions of chunks might justify a larger, code-specialized embedding model instead.

Q: How do you make sure the LLM doesn't hallucinate or make up an answer?

A: By grounding it: the system prompt explicitly instructs the model to answer using only the retrieved context and to cite sources by number. It's not a hard guarantee — no prompt fully eliminates hallucination — but restricting the model to real retrieved code snippets, and then only surfacing the citations it actually referenced, makes answers verifiable: a user can click through and check the exact code the answer is based on.

Q: What happens if no LLM API key is configured at all?

A: The RAG engine falls back to a local heuristic synthesis — it formats the top retrieved code chunk directly into a readable Markdown answer with its file path, line range, and a relevance score, so the tool degrades gracefully to 'here's the most relevant code I found' instead of failing outright.

Q: How do you keep answers from one repo leaking into another, since it's all one shared index?

A: Every chunk is tagged with a repo_name field in its Pinecone metadata at ingestion time, and every query applies a metadata equality filter on that field. It works today, but it's a soft, application-level boundary rather than a hard infrastructure boundary — a real multi-tenant version should move to Pinecone namespaces or separate indexes per tenant for a stronger guarantee.

12.3 Systems / Backend / Concurrency

Q: Why does ingestion run on a background thread instead of just an async function?

A: Because scanning, AST parsing, and embedding are CPU-bound, synchronous operations — running them directly inside an async request handler would block FastAPI's single event loop and freeze every other request (including status checks and the progress stream itself) until ingestion finished. A background daemon thread keeps that heavy work off the event loop entirely.

Q: How does progress get from that background thread into the async SSE stream?

A: The background thread never touches the event loop directly — it calls loop.call_soon_threadsafe(queue.put_nowait, event) on each connected client's asyncio.Queue, which is the standard safe way to schedule work onto an asyncio event loop from a different thread. The SSE endpoint's async generator is just awaiting on that queue and yields a new Server-Sent Event every time something lands in it.

Q: What is Server-Sent Events, and why use it here instead of WebSockets or polling?

A: SSE is a simple one-directional (server → client) streaming protocol over plain HTTP. It fits perfectly here because the browser only ever needs to receive progress updates — it never needs to send anything back over that connection — so SSE is simpler than a full-duplex WebSocket for this specific need, and far more efficient than the browser repeatedly polling a status endpoint.

Q: What happens if two people try to ingest a repo at the same time?

A: A module-level threading.Event (_ingest_running) acts as a simple global lock — a second ingest request while one is already running gets an HTTP 409 Conflict immediately, rather than the two ingestions racing and corrupting each other's progress or upserts.

Q: How would you make this handle more than one server instance (horizontal scaling)?

A: The current SSE client-tracking list and the ingest lock both live in one process's memory, so neither works correctly if you run multiple backend instances behind a load balancer. I'd move the client registry and locking to a shared layer — Redis Pub/Sub for broadcasting progress events across instances, and a distributed lock (or better, a real task queue) instead of an in-memory Event.

12.4 Comparison / Critical Thinking / Scaling

Q: What's the single biggest weakness of this project as it stands today?

A: The lack of real AST-based parsing for any language other than Python — since most real-world codebases are polyglot, that's a meaningful retrieval-quality gap for a large share of realistic use cases, and it's the fix I'd prioritize first (via tree-sitter).

Q: If I gave you a week to improve this project, what would you do?

A: Two things with the best effort-to-value ratio: fix clear_index() to only delete the active repo's vectors (a real correctness bug in a multi-repo setup), and add incremental re-indexing based on git diff so re-indexing a large repo after a small change doesn't require re-embedding everything from scratch.

Q: How would you scale this from a single-developer tool to a company-wide product?

A: Multi-tenant isolation (namespaces/auth per organization), event-driven re-indexing via GitHub webhooks instead of manual triggers, moving ingestion to a real task queue so it scales independently of the API server, and observability on retrieval quality and LLM cost — see §11.2 for the full breakdown. I'd also want caching for repeated common questions, since that cuts both cost and latency for the most frequently asked things about any given repo.

Q: Is vector search alone good enough for code search, or would you add anything?

A: Pure vector search is strong for conceptual/intent-based queries but can underperform on queries about exact identifiers, error strings, or config keys, where literal keyword matching actually wins. In a more mature version I'd combine dense vector search with a classic keyword index (a hybrid search setup) and add a reranking step on top of the merged candidates before they're sent to the LLM.

Q: Where does this project sit between a personal prototype and a production system?

A: It's a genuinely working, end-to-end RAG pipeline — ingestion, retrieval, grounded generation, and a real UI all function together — but it isn't production-hardened: no auth, a shared index with an index-wide clear bug, single-process assumptions in the SSE/locking layer, and no incremental indexing yet. I can speak concretely to exactly what I'd change to close that gap, which is what §10 and §11 cover.

13 One-Page Cheat Sheet (Skim This Right Before Walking In)

Fact	Value
What it is	RAG-based repository intelligence tool: semantic search + grounded Q&A; chatbot
Core pattern	Retrieval-Augmented Generation (RAG) — retrieve relevant code, then ground the LLM's answer in it
Ingestion steps	Scan → AST parse (Python) / sliding-window parse (other langs) → contextualize + chunk → embed → upsert to Pinecone
Embedding model	SentenceTransformers all-MiniLM-L6-v2 — 384 dimensions, runs locally
Vector DB	Pinecone serverless index, cosine similarity, metadata-filtered by repo_name for isolation
LLM providers	Gemini → Groq → OpenAI, in priority order, with a local heuristic fallback if none are configured
Citation mechanism	LLM instructed to cite with [n] markers; only referenced citations are surfaced to the UI
Backend	FastAPI + Uvicorn, single process — REST endpoints + SSE progress stream + background-thread ingestion
Concurrency bridge	loop.call_soon_threadsafe() moves progress events from the sync ingest thread into async SSE queues
Frontend	React 18 + TypeScript + Vite + TanStack Router/Query + Tailwind + Radix UI (shadcn-style)
Repo fetch strategy	GitHub API tree → zip archive download → shallow git clone, tried in that order
Biggest known limitation	Only Python gets true AST-based chunking; clear_index() wipes the whole shared index, not just the active repo

---

*Good luck — this project genuinely spans AI engineering (RAG, embeddings, vector search, multi-provider LLM integration), backend systems (async/threading concurrency, streaming), and frontend engineering. Lean into explaining **why** each design choice was made, and be ready to name its real limitations yourself — that's what will set you apart.*